De Babbelbox of Pieter Kelchtermans

scm.go, a Scheme interpreter in Go, as in SICP and lis.py

Posted on December 31, 2013 by pkelchte
This post is intended for people who know already a little bit of Go (http://golang.org/), and a little bit of Lisp (http://en.wikipedia.org/wiki/Lisp_(programming_language)). scm.go (https://gist.github.com/pkelchte/c2bd76b9f8f9cd603b3c) is a Scheme interpreter in Golang (http://golang.org/), inspired by implementations of Abelson and Sussman in Structure and Interpretation of Computer Programs (SICP) (http://mitpress.mit.edu/sicp/full-text/sicp/book/node77.html) and Norvig in lis.py (http://norvig.com/lispy.html). Thousands of other Lisp interpreters already exist, though not so many in Go (https://github.com/search?l=Go&q=lisp&ref=advsearch&type=Repositories). Few of them seemed to be designed with the simplicity of SICP or accessibility of lis.py in mind, and I wanted something for easy tinkering.

The goal was scm.go to be as simple and idiomatic Golang as possible. I hoped that clarity would then come for free, but feel that some documentation is still appropriate.

The interpreter consists of three parts: a parser for Lisp expressions (http://en.wikipedia.org/wiki/Lisp_(programming_language)#Syntax_and_semantics), a metacircular evaluator (http://books.google.be/books?id=68j6lEJjMQwC&lpg=PP1&dq=lisp%20programming%20manual&hl=nl&pg=PA13#v=onepage&q&f=false), and a Lisp environment (http://mitpress.mit.edu/sicp/full-text/sicp/book/node55.html) (constructed on top of Go objects).

The parser is stolen from Peter Norvig's lis.py. It takes an expression and parses it into a list, number, symbol, or any combination of those. Just as lispy's parser it expects one line of syntactically perfect Lisp from the user. It first splits the input in tokens, then parses the tokens into an internal representation of the expression. Expressions (http://www.schemers.org/Documents/Standards/R5RS/HTML/r5rs-Z-H-7.html#%_sec_4.1) are Lisp objects all the way down, but in scm.go, constant literals (http://www.schemers.org/Documents/Standards/R5RS/HTML/r5rs-Z-H-7.html#%_sec_4.1.2) (`number`s) are typed by `float64`, and variable references (http://www.schemers.org/Documents/Standards/R5RS/HTML/r5rs-Z-H-7.html#%_sec_4.1.1) (`symbol`s) by `string`. Although all these types share the common (empty) interface `scmer`, so differently typed values (in Go) can internally be stored in a list or passed along in the interpreter. Lists of Lisp objects are stored as slices of `scmer`.

The boring parser code can be found over here (https://gist.github.com/pkelchte/c2bd76b9f8f9cd603b3c#file-scm-go-L175-L220).

Once the expression is parsed and exists in memory, the metacircular evaluator can evaluate its contents. A case analysis in `eval` specifies how to determine an expressions value. If the expression is a list, and the first symbol in the list is not one of the six predefined special forms (`quote`, `if`, `set!`, `define`, `lambda`, `begin`), the interpreter assumes the expression is application. All subexpressions are first evaluated, then `apply` is called with the value of the first subexpression as operator (procedure), the others as operands (arguments).

```
1  /*
2   Eval / Apply
3  */
4
5  func eval(expression scmer, en *env) (value scmer) {
6      switch e := expression.(type) {
7      case number:
8          value = e
9      case symbol:
10         value = en.Find(e).vars[e]
11     case []scmer:
12         switch car, _ := e[0].(symbol); car {
13         case "quote":
14             value = e[1]
15         case "if":
16             if eval(e[1], en).(bool) {
17                 value = eval(e[2], en)
18             } else {
19                 value = eval(e[3], en)
20             }
21         case "set!":
22             v := e[1].(symbol)
23             en.Find(v).vars[v] = eval(e[2], en)
24             value = "ok"
25         case "define":
26             en.vars[e[1].(symbol)] = eval(e[2], en)
27             value = "ok"
28         case "lambda":
29             value = proc{e[1], e[2], en}
30         case "begin":
31             for _, i := range e[1:] {
32                 value = eval(i, en)
33             }
34         default:
35             operands := e[1:]
36             values := make([]scmer, len(operands))
37             for i, x := range operands {
38                 values[i] = eval(x, en)
39             }
40             value = apply(eval(e[0], en), values)
41         }
42     default:
43         log.Println("Unknown expression type - EVAL", e)
44     }
45     return
46 }
```

In `apply`, it is first determined if the procedure is a primitive or a compound procedure. The procedure is then applied to the list of arguments. A primitive procedure is a Go function with signature `func(...scmer) scmer`, that can be called there and then.

```
func apply(procedure scmer, args []scmer) (value scmer) {
    switch p := procedure.(type) {
    case func(...scmer) scmer:
        value = p(args...)
    case proc:
        en := &env{make(vars), p.en}
        switch params := p.params.(type) {
        case []scmer:
            for i, param := range params {
                en.vars[param.(symbol)] = args[i]
            }
        default:
            en.vars[params.(symbol)] = args
        }
        value = eval(p.body, en)
    default:
        log.Println("Unknown procedure type - APPLY", p)
    }
    return
}
```

The compound procedure case is a bit more involved. Because Go doesn't have lambda expressions like Python, I had to create a `struct` to represent a Lisp procedure: `proc`. Every `proc` object contains the expressions of the functions parameters and body, and a pointer to the environment in which it was created.

```
type proc struct {
    params, body scmer
    en           *env
}
```

When applying such a compound procedure, a new Lisp environment is constructed in which the parameters are variables bound to the arguments. The actual execution of the procedure will be the evaluation of its body in that environment. Because other symbols than the formal parameters can occur in the procedure's body, that environment points to the creators environment, where other symbols can be found.
Parameter-to-argument binding depends on the form of the parameter expression. If it is a list, parameters are bound to the arguments element by element. Otherwise, the parameter expression can be a symbol, then bound to the entire list of arguments, allowing e.g. defining `list` as `(lambda x x)`.

That leaves us with implementing environments, where Lisp objects can be bound to variable names (symbols). Each environment consists of a `map[symbol]scmer` named `vars` that provides this binding, and a pointer to its outer environment. The `Find` method returns the first environment where a given symbol is in the map, following the chain of outer environments.

```
/*
 Environments
*/

type vars map[symbol]scmer
type env struct {
    vars
    outer *env
}

func (e *env) Find(s symbol) *env {
    if _, ok := e.vars[s]; ok {
        return e
    } else {
        return e.outer.Find(s)
    }
}
```

For example expression `(define x 3)` parses to `["define" "x" 3]`, which eventually evaluates to a call `en.vars["x"] = 3`, effectively binding `3` to symbol `"x"` in the current environment.
Eventually, because in the define section of `eval`, there is another call to `eval` for the third element of the expression. In this case, that subexpression is number `3`, which evaluates to itself.
Evaluating `x` will now find symbol `"x"` in the current environment and return value `3`. To illustrate outer environments, let's define procedure `plusx` by evaluating `(define plusx (lambda (y) (+ y x)))`. Each time `plusx` is applied, e.g. evaluating `(plusx 4)`, a new environment is created in which only `"y"` is bound (to `4`), having as outer environment the current one. Upon evaluating the body `(+ y x)` in that environment, Find will not see `"x"` there, so looks for it in the outer environment, there finding the value `3` defined above.

Note that the symbol `"+"` is also not found in the new environment of the previous example… It is however among the symbols already in the current environment, because I put it there.

This environment, called `globalenv`, is initialized in `init()` at the start of the Go package, and contains definitions of a set of basic procedures: `+`, `-`, `equal?`, `cons`, `car`, `cdr`, all primitive procedures. An exception is `list`, defined as evaluating and parsing `"(lamda z z)"`, but you could just as well define it as a compound procedure `proc{symbol("z"), symbol("z"), &globalenv}`.

```
/*
 Primitives
*/

var globalenv env

func init() {
    globalenv = env{
        vars{ //aka an incomplete set of compiled-in functions
            "+": func(a ...scmer) scmer {
                v := a[0].(number)
                for _, i := range a[1:] {
                    v += i.(number)
                }
                return v
            },
            "-": func(a ...scmer) scmer {
                v := a[0].(number)
                for _, i := range a[1:] {
                    v -= i.(number)
                }
                return v
            },
            "*": func(a ...scmer) scmer {
                v := a[0].(number)
                for _, i := range a[1:] {
                    v *= i.(number)
                }
                return v
            },
            "/": func(a ...scmer) scmer {
                v := a[0].(number)
                for _, i := range a[1:] {
                    v /= i.(number)
                }
                return v
            },
            "<=": func(a ...scmer) scmer {
                return a[0].(number) <= a[1].(number)
            },
            "equal?": func(a ...scmer) scmer {
                return reflect.DeepEqual(a[0], a[1])
            },
            "cons": func(a ...scmer) scmer {
                switch car := a[0]; cdr := a[1].(type) {
                case []scmer:
                    return append([]scmer{car}, cdr...)
                default:
                    return []scmer{car, cdr}
                }
            },
            "car": func(a ...scmer) scmer {
                return a[0].([]scmer)[0]
            },
            "cdr": func(a ...scmer) scmer {
                return a[0].([]scmer)[1:]
            },
            "list": eval(read(
```

```
                "(lambda z z)"),
                &globalenv),
        },
        nil}
}
```

You can directly evaluate a fine number of expressions in this environment, or implement your own Lisp starting from the core provided here. So let's not stop yet and give the user an interactive shell, a read-eval-print loop (REPL (http://en.wikipedia.org/wiki/Read–eval–print_loop)). A simple one, where input expressions are line per line read and evaluated. A string representation of the result is printed for each evaluation, after which the loop starts reading the input again.

```
/*
 Interactivity
*/

func String(v scmer) string {
    switch v := v.(type) {
    case []scmer:
        l := make([]string, len(v))
        for i, x := range v {
            l[i] = String(x)
        }
        return "(" + strings.Join(l, " ") + ")"
    default:
        return fmt.Sprint(v)
    }
}

func Repl() {
    scanner := bufio.NewScanner(os.Stdin)
    for fmt.Print("> "); scanner.Scan(); fmt.Print("> ") {
        fmt.Println("==>", String(eval(read(scanner.Text()), &globalenv)))
    }
}
```

You can find the code as a whole in this github gist (https://gist.github.com/pkelchte/c2bd76b9f8f9cd603b3c)

Bookmark the permalink.

Create a free website or blog at WordPress.com. Do Not Sell or Share My Personal Information